

蓝桥杯30天算法冲刺集训



Day-6 搜索进阶与剪枝

DFS vs BFS

对比我们可以发现：广度优先搜索和深度优先搜索（尤其是非递归的栈实现）的基本过程很相似，都包含如下两个过程：

1. 判断边界：判断当前状态是否是目标状态，并进行相应处理
2. 扩展新状态：由当前状态（节点）出发，扩展出新的状态（节点）

它们的区别就是一个用队列实现和一个用栈实现，一个按层横向遍历，一个按列纵深遍历。数据结构的不同也导致了它们对待 **扩展出的新状态** 的处理策略不同。深度优先搜索会将所有的新状态压入栈中，采取 **先扩展出来的最后处理** 的策略。而广度优先搜索会将所有的新状态压入队列中，采取 **先扩展出来的先处理** 的策略。这也正是栈 **先入后出** 和队列 **先入先出** 的体现。

需要注意的是：DFS（深度优先搜索）是一种相对更常见的算法，大部分的题目都可以用 DFS 解决（而 BFS 广搜就不行了），但是大部分情况下，这都是骗分算法，很少会有爆搜为正解的题目。因为 DFS 的时间复杂度特别高。

但是，如果想掌握更高级的算法，DFS 必须要先熟练掌握！

搜索的解题步骤

1. 读清题目，理清状态是什么，状态包含哪些要素，初始状态是什么，目标状态是什么；
2. 建立搜索树，可以想象一下，或者在草稿纸上画，以帮助自己理解整个搜索树的构成，以及搜索的过程；
3. 明确从每一个状态出发可以转移到哪些新的状态（通过题目中允许的操作进行转移），转移是如何影响状态的，哪些要素发生了变化？
4. 需要进行回溯吗？状态的保存需要用全局变量吗？适合用广搜还是深搜？
5. 可以进行哪些剪枝，以减少不必要的搜索？
6. 按照代码框架，实现代码

搜索的一些优化

剪枝

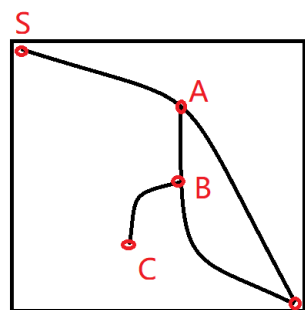
剪枝，顾名思义，就是通过一些判断，砍掉搜索树上不必要的子树。有时候，我们会发现某个结点对应的子树的状态都不是我们要的结果，那么我们其实没必要对这个分支进行搜索，砍掉这个子树，就是剪枝。

最常用的剪枝有三种，可行性剪枝、重复性剪枝、最优性剪枝。

可行性剪枝：在搜索过程中，一旦发现如果某些状态无论如何都不能找到最终的解，就可以将其“剪枝”了，比如越界操作、非法操作。一般通过条件判断来实现，如果新的状态节点是非法的，则不扩展该节点

重复性剪枝：对于某一些特定的搜索方式，一个方案可能会被搜索很多次，这样是没必要的。在实现上，一般通过一个记忆数组来记录搜索到目前为止哪些状态已经被搜过了，然后在搜索过程中，如果新的状态已经被搜过了，则不再扩展该状态节点。

最优性剪枝：对于求最优解的一类问题，通常可以用最优性剪枝，比如在求解迷宫最短路的时候，如果发现当前的步数已经超过了当前最优解，那从当前状态开始的搜索都是多余的，因为这样搜索下去永远都搜不到更优的解。通过这样的剪枝，可以省去大量冗余的计算，避免超时。在实现上，一般通过一个记忆数组来记录搜索到目前为止的最优解，然后在搜索过程中，如果新的状态已经不可能是最优解了，那再往下搜索肯定搜不到最优解，于是不再扩展该状态节点。



搜索从 S 到 E，假设先搜到了 $S \rightarrow A \rightarrow E$ 这条路径，其解为 30。

然后在 A 点回溯并搜完了 $A \rightarrow B \rightarrow E$ 这条路径，此时的 $S \rightarrow E$ 的解为 20，于是更新最优解为 20。

回溯到 B 点，然后搜到 C 点的时候，此时路径 $S \rightarrow A \rightarrow B \rightarrow C$ 的代价已经达到 20，那么再往下搜下去，结果肯定会大于 20，那肯定不会是最优解了，于是没必要再搜下去了，返回 E（于是剪枝了）

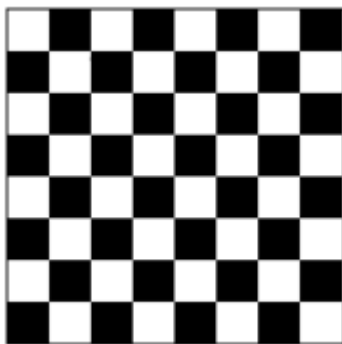
最优性剪枝的概念

其他的剪枝策略：

奇偶性剪枝

我们先来看一道题目：有一个 $n \times m$ 大小的迷宫。其中字符 S 表示起点，字符 D 表示出口，字符 X 表示墙壁，字符 . 表示平地。你需要从 S 出发走到 D，每次只能向上下左右相邻的位置移动，并且不能走出地图，也不能走进墙壁。每次移动消耗 1 时间，走过路都会塌陷，因此不能走回头路或者原地不动。现在已知出口的大门会在 T 时间打开，判断在 0 时间从起点出发能否逃离迷宫。数据范围 $n, m \leq 10, T \leq 50$ 。

我们只需要用 DFS 来搜索每条路线，并且只需搜到 T 时间就可以了（这是一个可行性剪枝）。但是仅仅这样也无法通过本题，还需考虑更多的剪枝。



如上图所示，将 $n \times m$ 的网格染成黑白两色。我们记每个格子的行数和列数之和 x ，如果 x 为偶数，那么格子就是白色，反之奇数时为黑色。容易发现相邻的两个格子的颜色肯定不一样，也就是说每走一步颜色都会不一样。更普遍的结论是：走奇数步会改变颜色，走偶数步颜色不变。

那么如果起点和终点的颜色一样，而 T 是奇数的话，就不可能逃离迷宫。同理，如果起点和终点的颜色不一样，而 T 是偶数的话，也不能逃离迷宫。遇到这两种情况时，就不用进行 *DFS* 了，直接输出 "NO"。

这样的剪枝就是奇偶性剪枝，本质上属于可行性剪枝。

蓝桥杯真题

[!NOTE]

前两题B站都有录播，也可以再参考B站的录播讲解。

填字母游戏（蓝桥杯C/C++2017A组国赛）

这个题目需要注意的是怎么用一个DFS就能判断出来在给定的字符串上面操作，到底是返回 0，-1 还是 1 的结果呢？

首先这种博弈类型的问题我们都是进行输赢的状态的转移的，也就是说如果状态下我找到一个 * 填入 L 或者 O 之后下一步再走DFS，回溯回来知道是一定输的，那么我们就是能赢的了。如果是平的，那么我们也是能平的，否则就是输的。

题目要让我们找最好的结果，那么我们找到能赢就返回，找到能平，就记录一下可以平，最后如果找不到能赢，就看是不是可以平，否则的话就是要输的。

我们还需要对状态进行一下记录，殊途同归，不论我们前面是怎么选取的 L 或者 O，只要到达这个状态，那么输赢就不会变换，所以需要记忆化搜索一下。

```
#include <bits/stdc++.h>
using namespace std;
map<string,int> mk;
char bk[]={'L','O'};
int dfs(string &s)
{
    if(mk.count(s)) return mk[s];
    if(s.find("*OL")!=-1 || s.find("L*L")!=-1 ||
s.find("LO*")!=-1) //check win
    {
        mk[s]=1;
        return 1;
    }
    if(s.find("*")==-1&& s.find("LOL")==-1) //check draw
    {
        mk[s]=0;
        return 0;
    }
    int draw = 0;
    for(auto &c:s)
    {
        if(c=='*')
        {
            for(int i=0;i<2;i++)
            {
                c=bk[i];
                int now=dfs(s);
                c='*';
                if(now==-1)
                {
                    mk[s]=1;
                    return 1;
                }
                else if(now==0) draw = 1;
            }
        }
    }
}
```

```

    }
    if(draw)
    {
        mk[s]=0;
        return 0;
    }
    mk[s]=-1;
    return -1;
}
void solve()
{
    string s;
    cin>>s;
    cout<<dfs(s)<<"\n";
}
int main()
{
    int T;
    cin>>T;
    while(T--) solve();
}

```

大胖子走迷宫（蓝桥杯C/C++2019A组国赛）

这题主要不同于一般题目的地方在于，小明的身材还会变化，也就是说最开始的时候呢是 5×5 的身材，后面会慢慢的变成 3×3 以至于变成 1×1 的普通的身材。

我们可以用BFS进行遍历，每次都通过一个四元组遍历也就是 $(x, y, step, size)$ 这个四元组，分别代表当前小明中心点的位置，小明走过的步数和小明当前的身材大小。

每次我们都有两种转移方式：

1. 小明以当前的身材向上下左右能走的地方去移动
2. 小明待在原地不动

然后我们需要注意到时间后，小明身材的变化，这里需要模拟一下。

最后只要能走到终点返回当前的 $step$ 就可以了。

```

#include <bits/stdc++.h>
using namespace std;
int n,k;
const int N = 305;
char a[N][N];
int vis[N][N][6];
int dx[]={0,0,-1,1},dy[]={-1,1,0,0};
struct node
{
    int x,y,step,size;
};
int check(int x,int y,int size)
{
    for(int i=x - size / 2;i<=x + size / 2;i++)
        for(int j=y - size / 2;j<=y + size / 2;j++)
        {
            if(i<1||j<1||i>n||j>n||a[i][j]=='*') return 0;
        }
    return !vis[x][y][size];
}
void bfs()
{
    queue<node> q;
    q.push({3,3,0,5});
    vis[3][3][5]=1;
    while(q.size())
    {
        node p = q.front();
        q.pop();
        if(p.x == n - 2 && p.y == n - 2)
        {
            cout<<p.step;
            break;
        }
        if(p.step>=k&&p.step<2*k) p.size=3;
        if(p.step>=2*k) p.size=1;
        for(int i=0;i<4;i++)
        {
            int xx = p.x + dx[i];
            int yy = p.y + dy[i];

```

```

        if(check(xx,yy,p.size))
        {
            vis[xx][yy][p.size]=1;
            q.push(node{xx,yy,p.step+1,p.size});
        }
    }
    if(p.size>1) q.push(node{p.x,p.y,p.step+1,p.size});
}
}
int main()
{
    cin>>n>>k;
    for(int i=1;i<=n;i++)
    for(int j=1;j<=n;j++) cin>>a[i][j];
    bfs();
}

```

最优旅行（蓝桥杯C/C++2019A组国赛）

这是一道结果填空题。首先题意是需要从北京出发，每个城市浏览一次（北京除外，可以多次停靠），最终回到北京。很明显是一道dfs+剪枝的题目，需要注意的是以下几点：

- 需要把时间换算成分钟，因为最后的结果是按照分钟计算的，而且需要注意24小时
- 小明决定从一个城市去往另一个城市时，他只会选择有直接高铁连接的城市，不会在中途换乘转车。
- 可以用字符串到int的映射来简化题目。
- 在计算运行时间cost和最终dfs中消耗时间的时候，需要注意的是隔天还是当天的车。

47373

```

#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

struct Node {
    int e; //终点
    int starttime,endtime; //起始时间，到达时间(转换为分钟表示)
}

```



```

int cost; // 运行时间
Node(int des,int startt,int endt,int time) {
    e = des;
    starttime = startt;
    endtime = endt;
    cost = time;
}
};
vector<vector<Node> > G(25); //存放所有班次信息
ll mintime = LONG_LONG_MAX; //最终结果
ll totaltime=0;
int vis[25]; // 去过哪些城市（除了北京只能去一次）

void dfs(int s,int endt,int len) { //起点城市 ， 到达此城市的时间，当前旅游过的城市长度
    if(s==1&&len>0) { //到达北京就要回溯（除了开始从北京出发）
        bool flag=true;
        for(int i=1; i<=20; i++) //检查是否旅游完毕,如果有在北京中转的情况，这种时候就会false
            if(vis[i]==0)
                flag=false;
        vis[1]=0; //北京始终允许反复访问（在北京多次中转）
        if(flag)//走遍所有地点（19个旅游城市+1个终点城市北京）
            mintime=min(mintime,totaltime);
        return;
    }
    for(int i=0; i<G[s].size(); i++) {
        Node r=G[s][i];
        if(vis[r.e]==0) { //目的城市没去过
            vis[r.e]=1;
            long long temp=totaltime;
            //time1: 此班车的运行时间
            totaltime+=r.cost;
            //time2: 班次间隔时间(24h内)
            if(s!=1&&r.starttime>endt) //此班车出发时间和上一班结束时间的关系
                totaltime+=r.starttime-endt;//休息时间（当日）
            if(s!=1&&r.starttime<endt)
                totaltime+=r.starttime-endt+1440; //跨日
            //time3: 从北京出发时的等车时间

```

```

        if(s==1) {
            if(r.starttime>720) //十二点出发
                totaltime+=r.starttime-720;
            else
                totaltime-=r.starttime-720+1440; //只有等到
明天才能出发了
        }
        //最优性剪枝:不剪枝会花跑很久
        if(totaltime>mintime) {
            totaltime=temp;
            continue;
        }

        dfs(r.e,r.endtime,len+1);
        vis[r.e]=0;
        totaltime=temp;
    }
}

int main() {

    map<string,int>mp;
    mp["北京"] = 1; mp["上海"] = 2;mp["广州"] = 3;mp["长沙"] =
4;mp["西安"] = 5;
    mp["杭州"] = 6; mp["济南"] = 7;mp["成都"] = 8;mp["南京"] =
9;mp["昆明"] = 10;
    mp["郑州"] = 11; mp["天津"] = 12;mp["太原"] = 13;mp["武汉"]
= 14;mp["重庆"] = 15;
    mp["南昌"] = 16; mp["长春"] = 17;mp["沈阳"] = 18;mp["贵阳"]
= 19;mp["福州"] = 20;

    string str;
    for(int i=1; i<=5; i++)
        cin>>str;
    for(int i=1; i<=132; i++) {
        cin>>str;
        string src,des;
        string s,t;
        cin>>src>>des>>s>>t;
        //换算时间为分钟

```

```

        int a,b;
        a=(s[0]-'0')*600 + (s[1]-'0')*60 + (s[3]-'0')*10 +
(s[4]-'0');
        b=(t[0]-'0')*600 + (t[1]-'0')*60 + (t[3]-'0')*10 +
(t[4]-'0');
        int cost; //运行时间
        if(a<b)
            cost=b-a;
        else
            cost=b-a+1440; //+1天
        //存入图中
        G[mp[src]].push_back(Node(mp[des],a,b,cost));
    }
    dfs(1,0,0); //从北京出发
    mintime+=1440*19; //加上19天的停留(如果经过北京中转,不会停留
24h)
    /*任意两趟班次之间:前一班的到达时间 到 下一班的发车时间间隔都是
小于24h
    例如:前一班15:30到,下一班16.30出发 需要休息一天+24h
    前一班15.30到,下一班08.30出发 也需要休息一天+24h
    */
    cout<<mintime;
    return 0;
}
//注意:班次信息为每天固定发车,即每天都会同一时间发同一班车
//ANS : 47373

```

习题

小蓝与迷宫

这个题目实际上还是贪心+搜索。

首先我们明白去拦截我们的人怎样做才能最优。也就是怎么做才能在一定能拦截我们的情况下,拦截我们。显然我们可以走的路径不是唯一的,那该怎么办呢。实际上直接都在终点等候即可。也就是说不论我们自身如何去走,拦截我们的人只需要在走最短路先到终点等候即可。

那这样的话做两遍BFS直接能够解决。一遍BFS从终点出发，找到所有拦截的人到终点的最小步数。默认这个步数为一个极大值，如果步数最终还是一个极大值，那么说明他无法到达终点，否则就记录下这个最小步数。

然后再一个BFS从当前人物出发，走到最终的终点，记录下来这个最小步数。然后都可以在这个最小步数之前到达终点的拦截的人都可以要到小蓝的签名。

C++代码：

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e3 + 10;
char a[N][N];
int sx, sy, ex, ey, vis[N][N], dx[] = {-1, 1, 0, 0}, dy[] = {0, 0, -1, 1}, d[N][N], ot, n, m, ans;
struct node
{
    int x, y, cur;
};
void bfs1(int x, int y)
{
    vis[x][y] = 1;
    queue<node> q;
    q.push(node{x, y, 0});
    while(q.size())
    {
        node p = q.front();
        q.pop();
        if(p.x == ex && p.y == ey)
        {
            ot = p.cur;
            break;
        }
        for(int i = 0; i < 4; i++)
        {
            int xx = p.x + dx[i];
            int yy = p.y + dy[i];
            if(xx >= 1 && xx <= n && yy >= 1 && yy <= m && !vis[xx][yy] && a[xx][yy] != 'T')
            {
```

```

        vis[xx][yy]=1;
        q.push(node{xx,yy,p.cur+1});
    }
}
}
}
void bfs2(int x,int y)
{
    vis[x][y]=1;
    queue<node> q;
    q.push(node{x,y,0});
    while(q.size())
    {
        node p=q.front();
        q.pop();
        d[p.x][p.y]=p.cur;
        for(int i=0;i<4;i++)
        {
            int xx=p.x + dx[i];
            int yy=p.y + dy[i];
            if(xx>=1&&xx<=n&&yy>=1&&yy<=m&&!vis[xx][yy]&&a[xx]
[yy]!='T')
            {
                vis[xx][yy]=1;
                q.push(node{xx,yy,p.cur+1});
            }
        }
    }
}
int main()
{
    cin>>n>>m;
    for(int i=1;i<=n;i++)
    for(int j=1;j<=m;j++)
    {
        cin>>a[i][j];
        if(a[i][j]=='S') sx=i,sy=j;
        if(a[i][j]=='E') ex=i,ey=j;
    }
}

```

```

memset(d,0x7f,sizeof(d));
bfs1(sx,sy);
memset(vis,0,sizeof(vis));
bfs2(ex,ey);

for(int i=1;i<=n;i++)
for(int j=1;j<=m;j++)
if(a[i][j]>='0'&&a[i][j]<='9')
{
    if(d[i][j]<=ot) ans+=a[i][j]-'0';
}
cout<<ans;
}

```

小红与字符串矩阵

题解：这题的核心是搜索+剪枝

这题是一个迷宫寻路算法，只不过需要按照它给定的路径行走，所以立马想到使用dfs。

和朴素dfs不同的地方是，这题不需要check一下是否越界（java和python好像需要判断，不然越界会错误），只需要看一下下一步是否是我们要走的字符串中对应步数的字符。now==6说明找到了整个tencent字符串，直接返回。需要注意的是，如果你的now是从0开始的，我们需要匹配的字符串要从第二个字母e开始 代码如下：

C++代码

```

#include <bits/stdc++.h>
using namespace std;

const int maxn = 1000 + 10;

string ten = "encent"; //需要注意，now从0开始的话直接从第二个字母
                        开始
int dx[] = {0,0,1,-1};
int dy[] = {1,-1,0,0};

char mp[maxn][maxn];
int ans = 0;

```

```

int n,m;

void dfs(int x,int y,int now){
    //第6步形成了tencent
    if(now == 6) {
        ans++;
        return;
    }

    for(int i=0;i<4;i++) {
        int xx = x + dx[i];
        int yy = y + dy[i];
        //如果下一个字符对应的的话，就继续dfs
        if(ten[now] == mp[xx][yy]) {
            dfs(xx,yy,now+1);
        }
    }
}

int main() {

    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    cin>>n>>m;
    for(int i=0;i<n;i++) {
        for(int j=0;j<m;j++) {
            cin>>mp[i][j];
        }
    }
    for(int i=0;i<n;i++) {
        for(int j=0;j<m;j++) {
            if(mp[i][j] == 't')
                dfs(i,j,0);
        }
    }
    cout<<ans;
    return 0;
}

```

